PROJECT REPORT

Linux Debugging System: Memory Leak Detection and Reporting Tool

Hackathon Date: 17 October 2026

Team Members: [Add names and roll numbers]

1. Abstract

This project implements a lightweight C utility that tracks selected dynamic memory allocations in a Linux program. It records allocation size and source location, marks tracked blocks as released when the corresponding wrapper is used, and reports allocations that remain outstanding at the end of a test. A text report is generated for review and documentation.

2. Problem Statement

In C programs, dynamic memory is often obtained at runtime. If a program no longer needs a block but does not release it, the block remains allocated until the process exits. Repeated leaks in long-running applications can increase memory consumption and degrade reliability. Developers need a practical way to identify unreleased allocations during testing.

3. Objectives

Track selected dynamic allocations; record allocation size and source location; track corresponding releases; identify outstanding tracked blocks; print a clear summary and save it to a report file; demonstrate both leak and clean scenarios on Linux.

4. Proposed Solution

The solution provides wrapper functions for malloc, calloc, and free. Successful allocations are recorded in a fixed-size table. When the tracked free wrapper receives a pointer, the matching record is marked inactive and the memory is released. At report time, all active records are listed as potential leaks.

5. System Design

Input: test mode (leak or clean) and optional report path. Processing: allocate memory, update tracking records, release selected blocks, inspect active records. Output: terminal summary and text report.

6. Technologies Used

C language; GCC compiler; Ubuntu/Linux; standard C library allocation functions; Git/GitHub for version control and collaboration.

7. Implementation Details

leak_tracker.h declares the API and macros. leak_tracker.c maintains allocation records and generates reports. main.c contains reproducible test modes. Source file and line number are captured through __FILE__ and __LINE__ macros.

8. Testing and Validation

Intentional leak: allocate two 100-byte blocks and release one; expected one outstanding block and 100 potential leaked bytes. Clean scenario: allocate an integer array and release it; expected zero outstanding tracked blocks. Additional tests include help, invalid option, and report-file output.

9. Expected Results

The intentional-leak test should identify the unreleased block and display its size and allocation location. The clean test should show zero outstanding tracked blocks. Reports should be saved in a readable text format. Run the executable before the presentation and replace expected results with observed results if they differ.

10. Limitations

The tracker only knows about allocations made through its wrapper API. It does not automatically inspect arbitrary binaries or intercept every direct call to malloc. It uses a fixed table of 4096 active records. A block remaining allocated at process exit is reported as a potential leak, because some programs intentionally retain memory until termination.

11. Responsible Implementation

The demo uses a small, intentional leak in a short-lived local process. It does not modify kernel memory, interfere with other users, or inspect unrelated processes. Test in a controlled development environment.

12. Future Enhancements

Add scalable tracking storage; detect duplicate frees and untracked frees more explicitly; add JSON/CSV reporting; optionally compare results with Valgrind; integrate the report command into the existing Linux Debugging System menu.

13. Conclusion

The prototype demonstrates the core debugging workflow of tracking allocations, matching releases, identifying outstanding blocks, and reporting results. It provides a foundation for further integration into a broader Linux debugging utility.